regaudit-fhe

Depth-Tracked Regulatory Audit Primitives
for Privacy-Preserving AI Audits

VaultBytes Innovations Ltd
b@vaultbytes.com | vaultbytes.com

2026-04-27 | version 0.0.7 | AGPL-3.0-or-later

Abstract

regaudit-fhe is an open-source Python library that provides six regulatory-audit primitives–fairness, drift, calibration, provenance, concordance, and cross-model disagreement–evaluable on encrypted inputs under CKKS at multiplicative depth six without bootstrapping. Each primitive is a depth-tracked slot-vector circuit exercised by both a plaintext reference model (`numpy`) and a TenSEAL CKKS backend. Outputs are wrapped in an Ed25519-signed audit envelope with canonical JSON, parameter-set hash, and per-input SHA-256 commitments. The library is positioned as a source of *technical evidence* that compliance workflows can ingest, not as a regulatory product. This whitepaper documents the problem the library addresses, the system architecture, the threat and leakage model, the six primitives, the CKKS parameter selection, the envelope-verification surface, and the limitations a reviewer needs to know before adopting it.

1. Problem

Regulators are converging on requirements that AI systems be audited at run-time for bias, drift, calibration, training-data provenance, challenger-model disagreement, and survival-model concordance. Examples include New York City Local Law 144 [1], the EU AI Act articles 10 and 15 [2], the Colorado AI Act [3], the U.S. CFPB algorithmic- discrimination guidance [4], FDA SaMD predetermined-change-control plans [5], ISO/IEC 23053 [6], OCC Supervisory Letter SR 11-7 [7], EMA AI guidance [8], and Basel-Committee model-risk and risk-data publications [9].

Each rule expects *evidence*: a numerical scalar, a pass/fail indicator, a calibrated set, or a top-K histogram. The data the rule applies to is, by definition, sensitive: protected-attribute indicators, per-row labels and predictions, PHI, or training-row identifiers. The natural way to comply–ship raw labels and attributes to a third-party auditor–collides with the GDPR, HIPAA, CCPA, and most internal data-handling policies. The natural way to preserve privacy–keep the data in-house–makes independent verification structurally weak.

regaudit-fhe fills the narrow gap between those two positions. It provides primitives that compute the regulator-relevant scalars under CKKS encryption, never decrypts per-row data, and emits a regulator-facing audit envelope that a third-party verifier can check without ever seeing the inputs. The library does not claim to discharge any regulator's specific compliance obligation; it produces *repeatable, signed, parameter-bound technical evidence* that compliance professionals can use as one input to that decision.

[1] NYC DCWP, *Local Law 144 of 2021 — Automated employment decision tools.*
[2] Regulation (EU) 2024/1689 of the European Parliament and of the Council, articles 10 (data governance) and 15 (accuracy, robustness, post-market monitoring).
[3] Colorado SB24-205, *Consumer Protections for Artificial Intelligence*, 2024.
[4] CFPB, *Circular 2022-03: Adverse action notification requirements in connection with credit decisions based on complex algorithms.*
[5] U.S. FDA, *Marketing submission recommendations for a predetermined change control plan for AI/ML-enabled device software functions*, final guidance, December 2024.
[6] ISO/IEC 23053:2022, *Framework for AI systems using machine learning.*
[7] Board of Governors of the Federal Reserve System and OCC, *Supervisory Guidance on Model Risk Management*, SR 11-7 / OCC 2011-12.
[8] EMA, *Reflection paper on the use of AI in the medicinal product lifecycle*, 2024.
[9] Basel Committee on Banking Supervision, *Principles for effective risk data aggregation and risk reporting*, BCBS 239, 2013; *Principles for the sound management of operational risk (revised)*, BCBS 261, 2021. Adopters should also consult any successor guidance specific to model risk in their jurisdiction.

2. Architecture

The library is organised into six layers, each of which can be substituted independently:

1. **Plaintext SlotVec model.** `regaudit_fhe._slot` implements a `numpy`-backed slot-vector with a strict multiplicative-depth tracker. Every operation (`mul_pt`, `mul_ct`, `mul_scalar`, `rotate`, `sum_all`, prefix-sum) increments the tracker; the constructor raises `DepthBudgetExceeded` if the tracked depth ever leaves the declared budget.

2. **Audit primitives.** Six modules (`egf_imss`, `etk_fpa_hbc`, `esc_cia`, `ecp_qssp`, `ew1_cdsf`, `ecmd_jps`) each expose a plaintext oracle and a depth-budgeted SlotVec circuit with identical semantics.

3. **TenSEAL CKKS backend.** Under the optional `[fhe]` extra, `regaudit_fhe.fhe` replicates the SlotVec algebra over TenSEAL ciphertexts. Each primitive has an `*_encrypted` variant that produces the same numerical output within CKKS noise tolerance.

4. **Audit envelope.** `regaudit_fhe.reports` wraps primitive outputs in a canonical-JSON envelope carrying a SHA-256 receipt, an Ed25519 signature, the parameter-set hash, and per-input SHA-256 commitments. An optional RFC 3161 timestamp-authority hook is included.

5. **JSON Schemas.** Thirteen Draft 2020-12 schemas (`schemas/`) describe every primitive input, every primitive output, and the envelope itself. The CLI and HTTP server validate every payload against the matching schema.

6. **Surfaces.** A `regaudit-fhe` command-line interface (audit, verify, schema, serve) and a hardened FastAPI HTTP server expose the same surface to clients (audited entities) and verifiers (regulators).

2.1. Encrypt, evaluate, decrypt

The standard run looks like this:

```
Subject  --[CKKS public key]--> Subject encrypts inputs locally
                                v
                              Operator runs audit_* primitive
                                v   (depth <= 6, no bootstrap)
                                Encrypted result + signed envelope
                                v
                              Auditor decrypts result
                                v
                              Plaintext-side threshold compare
                              + receipt verification
```

The Operator sees only ciphertexts and auditor-public scalars; the Auditor sees only the small set of decrypted aggregates the primitive exports. In the separated-role deployment, no single role sees both per-row plaintext inputs and operator-side ciphertext-evaluation state. When two or more roles are filled by the same legal entity (see §3 for the role definitions), the relevant separation must be re-established at the deployment layer (KMS partitioning, account boundaries, signed-evidence escrow).

3. Threat Model

Four roles participate in the audit pipeline. They MAY be the same legal entity but the cryptographic contract treats them as distinct.

Role	Holds	Trust
Subject	Plaintext per-row sensitive data.	Trusted to encrypt before sending.
Auditee	Model under audit; encryption keys.	Honest-but-curious.
Operator	Host running `regaudit_fhe.audit_*`.	Honest-but-curious.
Auditor	Ed25519 verifying-key chain; CKKS secret key when decrypting.	Honest.

3.1. Cryptographic guarantees

- **IND-CPA at 128-bit security** under Ring-LWE for the declared parameter set ($N = 2^{15}$, $\log Q \approx 240$, hybrid key switching dnum = 3). This is a parameter-level claim grounded in the HomomorphicEncryption.org standard tables [10] and the SEAL/TenSEAL recommended profiles; adopters should re-verify against an LWE/RLWE estimator (e.g. the Lattice Estimator) for their library versions and deployment profile before production use.
- **No bootstrapping.** Depth budgets are enforced both statically (the `CKKSParams` validator refuses chains that cannot evaluate the declared depth) and dynamically (the SlotVec tracker raises if a circuit exceeds its budget).
- **Tamper-evident envelope.** SHA-256 over canonical JSON bytes; reordering keys or whitespace produces a different digest.
- **Issuer authenticity.** Ed25519 signature over the same canonical body, plus an embedded `key_id` that verifiers can pin against a trusted-keys store.
- **Input binding.** `input_commitments` contain SHA-256 digests of canonicalised per-input vectors; a verifier holding the inputs can confirm the envelope describes those exact bytes.

3.2. Per-primitive public surface

Per-primitive tables enumerating which values are encrypted, which are auditor-public, and why each leakage is acceptable are in `docs/THREAT_MODEL.md` and reproduced in summary form in §4 below.

3.3. Out-of-scope attacks

Side channels in the host CPU, malicious Subjects who craft inputs to provoke false-negative breach decisions, malicious Auditors who leak decrypted aggregates, long-term cryptanalytic advances, and SHA-256 collisions are explicitly out of scope. They are deferred to the deployment layer (TEE, hardened hosts, contractual controls) and to standard collision-resistance assumptions.

4. Six Primitives

Every primitive ships a plaintext oracle, a depth-budgeted SlotVec circuit, an encrypted TenSEAL variant, and matching input/output JSON Schemas. The depth column shows observed depth on the TenSEAL backend; all six fit comfortably under the global $d = 6$ budget. The public API uses descriptive names (`audit_fairness`, `audit_drift`, ...); the internal module mnemonics (`egf_imss`, `ew1_cdsf`, etc.) are implementation details retained for stable file paths and are not exposed to callers.

Module	API	Depth	Use
`egf_imss`	`audit_fairness`	4	Demographic-parity, equal-opportunity, predictive-parity disparity scalars between two protected groups.
`etk_fpa_hbc`	`audit_provenance`	3	Top-K training-data provenance histogram via hash-bucket convolutional aggregator.
`esc_cia`	`audit_concordance`	4	Encrypted Harrell C-index over slot-shifted survival pairs.
`ecp_qssp`	`audit_calibration`	3	Conformal prediction-set bitmask via slot-packed quantile thresholds.
`ew1_cdsf`	`audit_drift`	2†	Cramer-von-Mises CDF L2-squared distance via prefix-sum + ct×ct square.
`ecmd_jps`	`audit_disagreement`	5	Pairwise variance across $M \geq 3$ deg-3 polynomial surrogate models.

† Logical SlotVec depth for `audit_drift` is 1 (rotation-based prefix sum); the TenSEAL backend declares 2 because it substitutes a depth-1 plaintext-matrix multiply for the prefix sum (TenSEAL CKKSVector has no public rotation primitive). Both numbers are reported in benchmarks under the **TenSEAL observed/declared depth** convention.

[10] HomomorphicEncryption.org, *Homomorphic Encryption Standard*, ver. 1.1, 2018, and subsequent table revisions.

4.1. Per-primitive notes

audit_fairness. Group cardinalities and label-positive counts are auditor-public (so reciprocals can be encoded as plaintext scalars and avoid an encrypted division). Disparity scalars and the boolean breach indicator are decrypted at the audit boundary.

audit_provenance. The hash function and seed are public, so the verifier can reproduce bucket assignments from row identifiers. Per-row attribution magnitudes never leave the encrypted domain.

audit_concordance. Two encrypted aggregates leave the boundary–concordant pairs and comparable pairs–and the C-index ratio is computed plaintext-side. PHI never leaves the encrypted domain.

audit_calibration. Per-class quantile thresholds are auditor-public (they are the calibration-set output). The slot permutation that routes encrypted scores into matched class slots is class-invariant and therefore reveals nothing per test point.

audit_drift. The Cramer-von-Mises L2-squared distance is chosen instead of strict W1 to avoid an encrypted absolute-value approximation. It preserves the key equality condition used in drift detection – zero iff the empirical distributions agree – and provides a practical shift-sensitive statistic while avoiding the encrypted absolute-value approximation that a strict W1 evaluator requires. It fits in a single multiplicative level.

audit_disagreement. Each of $M \geq 3$ model versions is represented by an auditor-public deg-3 polynomial surrogate; the encrypted input vector never leaves the domain.

5. CKKS Parameterization

`regaudit_fhe.fhe.params.CKKSParams` is the canonical parameter set; constructing one runs every validation rule the library promises to enforce.

Field	Default	Semantics
`ring_dim`	2^{15}	Power of two; smaller rings cannot hold the full chain.
`multiplicative_depth`	6	Must lie in $[1, 6]$; the audit primitives are bounded above by six.
`scaling_mod_size`	40	Bit-size of every interior prime.
`first_mod_size`	60	Bit-size of the leading and trailing primes.
`security_level`	`HEStd_128_classic`	At least 128-bit IND-CPA.
`coeff_mod_bit_sizes`	$\langle 60, 40, 40, 40, 40, 40, 40, 60 \rangle$	8 primes; supports six multiplications.
`rotation_steps`	$\pm 2^k$ ladder	Halevi-Shoup ladder; minimal key set.
`precision_loss_bound`	10^{-2}	Worst-case rescale-side loss bound.

5.1. Validation rules

The constructor enforces and rejects with `ParameterValidationError`:

1. Ring dimension is power-of-two and $\geq 2^{14}$.
2. Security level is at least 128-bit IND-CPA.
3. Modulus chain length is $\geq d + 2$ for the declared depth.
4. $\sum_i \log_2 q_i$ is within the SEAL-validated bound for the chosen ring dimension at 128-bit security.
5. `first_mod_size` $\geq$ `scaling_mod_size`; every interior prime is $\geq$ `scaling_mod_size`; first and last primes are $\geq$ `first_mod_size`.
6. No bootstrapping is required: the chain is long enough.
7. The rotation key set is exactly the Halevi-Shoup ladder plus any explicitly acknowledged extras.
8. Worst-case precision loss $d \cdot 2^{-\texttt{scaling_mod_size}} \leq$ `precision_loss_bound`.

5.2. Backend choice

The repository ships exactly one encrypted backend: TenSEAL ($\geq$ 0.3.14). It is loaded only when the user installs `regaudit-fhe[fhe]`. OpenFHE is on the roadmap but not included in this repository; the design note lives at `docs/roadmap/openfhe_backend.md`. Switching backends is explicit; the `parameter_set.backend` field of the audit envelope reads `tenseal-ckks` on the current build and would read `openfhe-ckks` on a future OpenFHE-backed build.

5.3. Real measurements

The benchmark suite at `benchmarks/bench_fhe.py` produces machine-readable JSON under `benchmarks/results/`. Headline numbers at $N = 2^{15}$:

Primitive	Depth obs/decl	Runtime	RAM	Max abs error
fairness	1 / 4	0.58 s	2.7 GB	7.2×10^{-7}
provenance	1 / 3	1.52 s	2.7 GB	1.9×10^{-5}
drift	2 / 2	0.39 s	2.8 GB	1.1×10^{-5}
disagreement	3 / 5	0.35 s	2.7 GB	1.4×10^{-8}

Reproducibility metadata

Field	Value
CPU	Apple M3 Max
RAM	36 GB unified memory
OS	macOS 26.3.1 (Darwin 25.3.0)
Python	3.12.11
NumPy	2.4.4
TenSEAL	0.3.16
SEAL	shipped inside TenSEAL 0.3.16 (Microsoft SEAL 4.x)
Threads	default (TenSEAL/SEAL single-threaded execution)
Repetitions	3 per primitive per ring
Statistic reported	median of the three repetitions
Commit	`dab476a` (`regaudit-fhe` v0.0.7)

Command (literal ASCII; copy-paste from the PDF preserves the double hyphens):

```
python benchmarks/bench_fhe.py --rings 14 15 --reps 3 --flip-trials 10
```

The threshold-flip rate – the fraction of trials in which CKKS noise flips a boolean breach decision against the plaintext oracle – is 0.00% across both rings tested (2^{14} and 2^{15}). Adopters should re-run `benchmarks/bench_fhe.py` on their target hardware before publishing comparable numbers; runtime is sensitive to CPU SIMD width and library versions.

Security estimates above are parameter-level claims and should be revalidated by adopters using their chosen HE security estimator, deployment profile, and library versions.

6. Envelope Verification

The audit envelope is the regulator-facing artefact every audit produces. A verifier with the issuer's public key can decide three questions without evaluating the audit circuit, in time linear in the envelope size:

Q1. **Was it produced by a trusted issuer?** Ed25519 signature over the canonical body, plus an embedded `key_id` that the verifier matches against a trusted-keys store.

Q2. **Has it been modified since issuance?** SHA-256 over the same canonical body.

Q3. **Were the encrypted inputs the issuer claims to have audited the same inputs the verifier received?** `input_commitments[i].sha256` digests of canonicalised per-input vectors.

6.1. Canonical JSON rule

Every signed body is serialised with `json.dumps(payload, sort_keys=True, separators=(",", ":"), ensure_ascii=False)` and encoded as UTF-8 before hashing or signing. Verifiers re-serialising with the same rule MUST produce byte-identical output. Producers that emit non-canonical JSON–extra whitespace, mixed key order, escaped Unicode–will fail verification.

6.2. Envelope shape

```
{
  "schema":              "regaudit-fhe.report.v1",
  "schema_version":      "regaudit-fhe.report.v1",
  "algorithm_version":   "0.0.7",
  "primitive":           "fairness",
  "backend":             "tenseal-ckks",
  "parameter_set":       { ... validated CKKSParams ... },
  "parameter_set_hash": "sha256:6b8aedc173e2c94e...",
  "regulations":         ["NYC_LL144", "EU_AI_ACT_ART10", ...],
  "result":              { ... primitive-specific output ... },
  "input_commitments":   [{"name": "y_true",  "sha256": "..."},
                          {"name": "y_pred",  "sha256": "..."}],
  "depth_budget":        {"declared": 6, "consumed": 4},
  "issued_at":           "2026-04-27T12:34:56+00:00",
  "issuer":              "example-vendor",
  "receipt": {
    "sha256":          "sha256:9f3c...",
    "signature_alg":   "Ed25519",
    "signature_b64":   "MEUCIQDX...",
    "key_id":          "auditor-key-2026-04",
    "public_key_pem": "-----BEGIN PUBLIC KEY-----\n..."
  },
  "timestamp":           null
}
```

6.3. Verifier checklist

A regulator-side verifier must run every step of the following before accepting an envelope:

1. `schema` equals `"regaudit-fhe.report.v1"`; the `algorithm_version` is on the verifier's accepted list.
2. `parameter_set_hash` is one of the verifier's pinned parameter-set hashes.
3. Canonical SHA-256 of the body equals `receipt.sha256`.
4. Ed25519 verify of `receipt.signature_b64` against `receipt.public_key_pem` succeeds.
5. `receipt.key_id` is on the verifier's trusted-keys list AND the embedded `public_key_pem` matches the pinned value for that `key_id`.
6. If `timestamp` is present, the TSA token verifies against the verifier's trusted TSA root.
7. For each input the verifier can reproduce, the canonical hash matches `input_commitments[i].sha256`.

The library function `regaudit_fhe.verify_envelope(env, trusted_keys=...)` returns a structured outcome covering steps 3-5. The remaining steps are policy-side and live in the verifier's code.

7. Limitations

- **Pre-alpha.** The package's PyPI classifier reads `Development Status :: 2 - Pre-Alpha`. Public APIs may change between minor versions.
- **Plaintext is the default execution path.** Encryption happens only when the caller explicitly imports `regaudit_fhe.fhe` and supplies a CKKS context.
- **Deg-3 polynomials.** The sign and reciprocal polynomials shipped in the open-source release are textbook deg-3 minimax fits with a $\sim$5% worst-case relative error on their declared input ranges. Calibrated higher-degree variants are not in this release.

- **TenSEAL-specific depth penalty for prefix-sum.** TenSEAL's CKKSVector lacks a public rotation primitive, so the drift primitive uses a depth-1 plaintext-matrix multiply for the CDF prefix-sum instead of a depth-0 rotation tree. The OpenFHE target backend would not pay this penalty.
- **No regulatory endorsement.** The library produces technical evidence only. No regulator has endorsed the audit envelope as a recognised compliance artefact.
- **No independent cryptography or security review.** The library has not yet been reviewed by an independent cryptography auditor or pen-test team.
- **HTTP server is a reference deployment.** It is not a privacy boundary by itself. Production deployments must terminate TLS at the ingress, hold the CKKS secret key in a separate KMS / HSM account, and apply the controls listed in `docs/DEPLOYMENT.md`.
- **No malicious-Operator defence beyond signing.** Operators that control the signing key entirely are out of scope; defence relies on KMS-backed signing and key-chain rotation handled at the deployment layer.
- **No post-quantum certification claimed.** The library relies on standard Ring-LWE assumptions as used in CKKS parameterizations. Production adopters should obtain independent cryptographic review for any PQC-sensitive deployment.
- **N=2^{16} benchmarks not in the default matrix.** The benchmark harness supports it via `--rings 16`, but the published numbers cover 2^{14} and 2^{15} only.

References and Further Reading

The repository ships the following companion documents:

- Per-primitive specifications – `docs/specs/`
- Threat / leakage model – `docs/THREAT_MODEL.md`
- Regulator-facing scope statement – `docs/COMPLIANCE.md`
- Deployment guide – `docs/DEPLOYMENT.md`
- Supply-chain controls – `docs/SUPPLY_CHAIN.md`
- OpenFHE roadmap note – `docs/roadmap/openfhe_backend.md`
- Reproducible benchmark harness – `benchmarks/bench_fhe.py`; results are machine-readable JSON under `benchmarks/results/`.

---

regaudit-fhe is licensed under AGPL-3.0-or-later.
Source: `github.com/BAder82t/regaudit-fhe`.
PyPI: `pypi.org/project/regaudit-fhe`.
Maintained by VaultBytes Innovations Ltd; contact `b@vaultbytes.com`.